

IMPLEMENTASI ALGORITMA STRUKTUR DATA PADA SISTEM PEMESANAN TIKET BIOSKOP

Disusun untuk Memenuhi Tugas Mata Kuliah Algoritma & Struktur Data

Dosen Pengampu:

Taufik Ridwan, M.T.



Disusun oleh:

Kelompok 2

Muhammad Fauzi Akbar	(2510631250009)
Fatih Askar Mubina	(2510631250060)
Putri Aulia	(2510631250021)
Kireina Oktaviani	(2510631250066)

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
2026**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan program Sistem Tiket Bioskop ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai dokumentasi dari program yang telah dibuat menggunakan bahasa pemrograman Java. Program ini merupakan implementasi dari berbagai konsep struktur data dan algoritma, meliputi penggunaan array, sorting, searching, serta pengelolaan file. Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami implementasi program manajemen tiket bioskop berbasis Java.

Karawang, 25 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I	
PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Tujuan.....	1
BAB II	
ANALISIS KEBUTUHAN.....	2
2.1. Analisis Fitur Sistem.....	2
2.1.1. Fitur Tambah Data Tiket.....	2
2.1.2. Fitur Menampilkan Seluruh Data.....	3
2.1.3. Fitur Edit Data.....	3
2.1.4. Fitur Hapus Data.....	4
2.1.5. Fitur Update Status.....	4
2.1.6. Fitur Searching Data.....	4
2.1.6.1. Pencarian Berdasarkan ID.....	4
2.1.6.2. Pencarian Berdasarkan Nama.....	4
2.1.6.3. Pencarian Berdasarkan Film.....	5
2.1.7. Fitur Sorting Data.....	5
2.1.7.1. Sorting Berdasarkan Nama.....	5
2.1.7.2. Sorting Berdasarkan ID.....	5
2.1.7.3. Sorting Berdasarkan Jumlah Tiket.....	5
2.1.8. Fitur Statistik dan Counter Data.....	5
2.1.9. Fitur Simpan dan Load Data.....	6
2.2. Analisis Struktur Data.....	6
2.2.1. Array.....	6
BAB III	
IMPLEMENTASI.....	8
3.1. Struktur Data dan Konsep Dasar Program.....	8
3.2. Penjelasan Class dan Deklarasi Variabel.....	8
3.3. Penjelasan Load dan Save Data.....	9
3.4. Penjelasan Data Awal dan Main Program & Penjelasan Fitur Tambah dan Tampil Data.....	11
3.5. Penjelasan Sorting dan Searching.....	13
3.6. Penjelasan Hapus Data.....	16
3.7. Penjelasan Statistik dan Counter Data.....	17
3.8. Penjelasan Edit dan Update Status.....	18
BAB IV	
ANALISIS KOMPLEKSITAS.....	20
4.1. Analisis Kompleksitas Program.....	20

4.2. Tabel Analisis Kompleksitas.....	20
4.3. Penjelasan Kompleksitas.....	22
4.3.1. Kompleksitas Penambahan Data.....	22
4.3.2. Kompleksitas Pengurutan Data.....	22
4.3.2.1. Selection Sort pada <code>urutNama()</code>	22
4.3.2.2. Bubble Sort pada <code>urutID()</code> dan <code>urutTerbanyak()</code>	23
4.3.3. Kompleksitas Pencarian Data.....	23
4.3.3.1. Binary Search (<code>cariID()</code>).....	23
4.3.3.2. Linear Search (<code>cariNama()</code> , <code>cariFilm()</code> , <code>edit()</code> , <code>updateStatus()</code>)..	24
4.3.4. Kompleksitas Penghapusan Data.....	24
4.3.5. Kompleksitas Pembacaan dan Penyimpanan File.....	25
4.3.5.1. <code>loadData()</code>	25
4.3.5.2. <code>simpanData()</code>	25
4.3.6. Analisis Space Complexity.....	25
BAB V	
PENGUJIAN.....	27
5.1. Tabel Rencana Pengujian (Test Matrix).....	27
5.2. Dokumentasi Eksekusi Skenario Pengujian.....	30
5.2.1. TC-01: Inisialisasi Data Awal (Load File).....	30
5.2.2. TC-02: Menambahkan Tiket Baru (Menu 1).....	31
5.2.3. TC-03: Menampilkan Seluruh Data (Menu 2).....	32
5.2.4. TC-04: Mengurutkan Nama Pemesan (Menu 3).....	33
5.2.5. TC-05: Mengurutkan ID Tiket (Menu 4).....	35
5.2.6. TC-06: Mengurutkan Tiket Terbanyak (Menu 5).....	36
5.2.7. TC-07: Pencarian ID Tiket via Binary Search (Menu 6).....	37
5.2.8. TC-08: Pencarian Nama via Linear Search (Menu 7).....	38
5.2.9. TC-09: Pencarian Multi-Data Berdasarkan Judul Film (Menu 8)....	39
5.2.10. TC-10: Penghapusan Elemen Permanen / Hard Delete (Menu 9)...	40
5.2.11. TC-11: Menampilkan Hasil Statistik Penjualan (Menu 10).....	42
5.2.12. TC-12: Mengubah Informasi Tiket / Edit Data (Menu 11).....	43
5.2.13. TC-13: Manajemen Flag Soft Delete / Update Status (Menu 12).44	
5.2.14. TC-14: Monitoring Kapasitas Memori Array (Menu 13).....	46
5.2.15. TC-15: Terminasi Program dan Sinkronisasi File Eksternal (Menu	
0).....	47
BAB VI	
KESIMPULAN.....	48

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi yang pesat mendorong berbagai sektor industri untuk mengadopsi sistem berbasis komputer guna meningkatkan efisiensi operasional. Salah satu sektor yang dapat memanfaatkan teknologi ini adalah industri hiburan, khususnya bioskop. Pengelolaan tiket secara manual memiliki banyak keterbatasan, seperti potensi kesalahan pencatatan, kesulitan dalam pencarian data, serta tidak adanya sistem pengurutan yang terstruktur.

Berdasarkan permasalahan tersebut, dikembangkanlah Sistem Tiket Bioskop berbasis Java yang mampu mengelola data pemesanan tiket secara digital. Sistem ini memanfaatkan konsep-konsep dasar pemrograman seperti array untuk penyimpanan data, algoritma sorting untuk pengurutan, algoritma searching untuk pencarian data, serta file I/O untuk persistensi data.

1.2. Tujuan

Tujuan dari pembuatan program Sistem Tiket Bioskop ini adalah:

1. Mengimplementasikan konsep array dalam bahasa Java sebagai struktur penyimpanan data tiket bioskop.
2. Menerapkan algoritma pengurutan (sorting) yaitu Selection Sort dan Bubble Sort untuk mengurutkan data berdasarkan nama, ID, maupun jumlah tiket.
3. Menerapkan algoritma pencarian (searching) yaitu Binary Search dan Linear Search untuk menemukan data secara efisien.
4. Mengimplementasikan fitur manajemen data lengkap meliputi operasi tambah, tampil, edit, hapus, dan update status.
5. Menyediakan fitur persistensi data menggunakan file teks (.txt) agar data tidak hilang saat program ditutup.
6. Menyediakan fitur statistik dan counter data untuk pemantauan kapasitas array

BAB II

ANALISIS KEBUTUHAN

2.1. Analisis Fitur Sistem

Sistem yang dibuat merupakan aplikasi pengelolaan tiket bioskop berbasis console menggunakan bahasa pemrograman Java. Program ini dirancang untuk membantu pengguna dalam mengelola data pemesanan tiket bioskop secara sederhana menggunakan konsep struktur data array dan algoritma dasar seperti searching serta sorting.

Data yang disimpan dalam sistem meliputi ID tiket, nama pemesan, judul film, jumlah tiket, dan status data. Selain itu, sistem juga mendukung penyimpanan data ke file teks sehingga data tetap tersimpan walaupun program ditutup.

Berikut merupakan fitur-fitur yang terdapat pada sistem.

2.1.1. Fitur Tambah Data Tiket

Fitur tambah data digunakan untuk memasukkan data pemesanan tiket bioskop baru ke dalam sistem. Pada saat fitur dijalankan, pengguna diminta menginput beberapa informasi seperti ID tiket, nama pemesan, judul film, dan jumlah tiket yang dibeli.

Setelah data dimasukkan, sistem akan membuat objek baru dari class `Tiket` kemudian menyimpannya ke dalam array data. Variabel `jumlahData` akan bertambah secara otomatis sesuai jumlah data yang berhasil disimpan.

Fitur ini berfungsi sebagai proses Create pada konsep CRUD (Create, Read, Update, Delete). Dengan adanya fitur tambah data, pengguna dapat memperbarui isi data tiket bioskop kapan saja sesuai kebutuhan.

2.1.2. Fitur Menampilkan Seluruh Data

Fitur tampil data digunakan untuk menampilkan seluruh data tiket bioskop yang tersimpan di dalam array. Data ditampilkan dalam bentuk tabel agar lebih mudah dibaca oleh pengguna.

Informasi yang ditampilkan meliputi:

- ID tiket
- Nama pemesan
- Judul film
- Jumlah tiket
- Status data

Status data dibedakan menjadi dua, yaitu aktif dan nonaktif. Status tersebut berasal dari variabel boolean `isDeleted`. Jika bernilai `false`, maka data dianggap aktif, sedangkan jika bernilai `true`, data dianggap nonaktif.

2.1.3. Fitur Edit Data

Fitur edit data digunakan untuk memperbarui data yang sudah tersimpan sebelumnya. Proses edit dilakukan berdasarkan ID tiket yang dimasukkan oleh pengguna. Ketika ID ditemukan, pengguna dapat mengganti nama pemesan, judul film, maupun jumlah tiket. Setelah proses selesai, data lama akan diperbarui dengan data baru yang telah dimasukkan.

2.1.4. Fitur Hapus Data

Fitur hapus data digunakan untuk menghapus data tiket berdasarkan ID tertentu. Pada sistem ini, metode yang digunakan adalah hard delete, yaitu data benar-benar dihapus dari array. Saat data dihapus, seluruh elemen setelah data tersebut akan digeser ke kiri agar tidak terdapat slot kosong di tengah array. Setelah itu, jumlah data akan dikurangi satu.

2.1.5. Fitur Update Status

Selain fitur hapus permanen, sistem juga memiliki fitur update status yang digunakan untuk mengubah status data menjadi aktif atau nonaktif. Fitur ini bekerja dengan memanfaatkan variabel boolean `isDeleted`. Jika status sebelumnya aktif, maka sistem akan mengubahnya menjadi nonaktif, dan sebaliknya.

Konsep ini menyerupai soft delete karena data tidak benar-benar dihapus dari sistem, melainkan hanya dinonaktifkan sementara. Dengan cara ini, data masih dapat disimpan dan digunakan kembali jika diperlukan.

2.1.6. Fitur Searching Data

2.1.6.1. Pencarian Berdasarkan ID

Pencarian ID menggunakan algoritma Binary Search. Sebelum proses pencarian dilakukan, data terlebih dahulu diurutkan berdasarkan ID menggunakan Bubble Sort. Binary Search bekerja dengan mencari data dari posisi tengah array sehingga proses pencarian menjadi lebih cepat dibandingkan pencarian biasa.

2.1.6.2. Pencarian Berdasarkan Nama

Pencarian nama menggunakan algoritma Linear Search. Sistem akan memeriksa data satu per satu dari awal hingga akhir array sampai nama yang dicari ditemukan.

2.1.6.3. Pencarian Berdasarkan Film

Fitur pencarian film digunakan untuk menampilkan seluruh data pemesan berdasarkan judul film tertentu. Sistem akan menampilkan semua data yang memiliki judul film yang sama. Fitur ini berguna untuk mengetahui daftar penonton dari suatu film tertentu.

2.1.7. Fitur Sorting Data

2.1.7.1. Sorting Berdasarkan Nama

Pengurutan nama menggunakan algoritma Selection Sort dengan urutan alfabet A-Z. Sistem akan mencari data dengan nama terkecil kemudian menukarnya dengan posisi awal array secara bertahap.

2.1.7.2. Sorting Berdasarkan ID

Pengurutan ID menggunakan algoritma Bubble Sort secara ascending. Sistem akan membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah.

2.1.7.3. Sorting Berdasarkan Jumlah Tiket

Sistem juga dapat mengurutkan data berdasarkan jumlah tiket terbanyak menggunakan Bubble Sort descending.

2.1.8. Fitur Statistik dan Counter Data

Sistem menyediakan fitur statistik untuk menghitung jumlah data aktif dan total seluruh tiket yang tersimpan di dalam sistem.

Selain itu, terdapat fitur counter data array yang menampilkan:

- Kapasitas maksimal array
- Jumlah slot terisi
- Jumlah data aktif

- Jumlah data nonaktif
- Sisa slot kosong

Fitur ini membantu pengguna mengetahui kondisi penyimpanan data dalam array sehingga penggunaan memori dapat dipantau dengan lebih baik.

2.1.9. Fitur Simpan dan Load Data

Sistem mendukung penyimpanan data ke dalam file teks bernama `tiket.txt`. Proses penyimpanan dilakukan menggunakan `FileWriter` dan `PrintWriter`. Ketika program dijalankan kembali, sistem akan membaca file tersebut menggunakan `FileReader` dan `BufferedReader`. Data kemudian dimasukkan kembali ke dalam array.

Fitur ini sangat penting karena memungkinkan data tetap tersimpan walaupun program telah ditutup. Dengan demikian, pengguna tidak perlu memasukkan ulang seluruh data setiap kali program dijalankan.

2.2. Analisis Struktur Data

2.2.1. Array

Array adalah struktur data yang menyimpan sekumpulan elemen bertipe sama dalam lokasi memori yang berurutan dan dapat diakses menggunakan indeks. Array bersifat statis, artinya ukurannya ditentukan saat deklarasi dan tidak dapat diubah secara dinamis.

Dalam program ini, array digunakan untuk menyimpan objek bertipe `Tiket` dengan kapasitas maksimal 100 elemen. Setiap elemen array diakses melalui indeks integer mulai dari 0 hingga (`jumlahData` - 1). Variabel `jumlahData` digunakan sebagai penunjuk jumlah slot yang telah terisi.

```
static Tiket[] data = new Tiket[100];  
static int jumlahData = 0;
```

Pada sistem ini, struktur data utama yang digunakan adalah array objek dengan class sebagai representasi data tiket bioskop. Struktur data dipilih karena sesuai dengan kebutuhan program yang masih berskala sederhana dan mudah dipahami.

BAB III

IMPLEMENTASI

3.1. Struktur Data dan Konsep Dasar Program.

```
1  import java.util.Scanner;
2  import java.io.*;
3
4  class Tiket {
5      int id;
6      String nama;
7      String film;
8      int jumlah;
9      boolean isDeleted;
10 }
```

Pada program sistem tiket bioskop ini digunakan beberapa struktur data dan konsep dasar pemrograman. Struktur data utama yang digunakan adalah array of object, yaitu array data [] yang berisi objek bertipe Tiket. Array digunakan untuk menyimpan banyak data tiket dalam satu variabel dengan kapasitas maksimal 100 data. Sedangkan object Tiket digunakan untuk merepresentasikan satu data pemesanan tiket yang memiliki atribut id, nama, film, jumlah, dan isDeleted. Selain array, program juga menggunakan class sebagai blueprint objek, sehingga setiap data tiket memiliki struktur yang sama. Program ini juga memakai Scanner untuk menerima input dari user serta file handling menggunakan BufferedReader, FileReader, PrintWriter, dan FileWriter untuk menyimpan serta membaca data dari file TXT.

3.2. Penjelasan Class dan Deklarasi Variabel

```
11 public Tiket(int id, String nama, String film, int jumlah) {
12     this.id = id;
13     this.nama = nama;
14     this.film = film;
15     this.jumlah = jumlah;
16     this.isDeleted = false;
17 }
18 }
19
20 public class Sistemtiketbioskopfixx {
21
22     static Tiket[] data = new Tiket[100];
23     static int jumlahData = 0;
24     static Scanner sc = new Scanner(System.in);
```

Bagian awal program memiliki class Tiket, yang berfungsi sebagai model data untuk tiket bioskop. Ada atribut id yang dapat digunakan untuk menyimpan nomor tiket, nama untuk nama pemesan, judul film, jumlah, dan isDeleted untuk menunjukkan apakah data aktif atau nonaktif. Saat object dibuat, seluruh data langsung dapat diisi sekaligus dengan menggunakan Constructor Tiket(). Dibandingkan dengan menggunakan banyak variabel terpisah, data menjadi lebih terstruktur dan mudah dikelola dengan class ini. Setelah class Tiket, program memiliki deklarasi variabel global seperti Tiket[] data, jumlahData, dan Scanner sc. Seluruh objek tiket disimpan dalam array data, sedangkan jumlahData menghitung jumlah data yang sudah terisi di array. Scanner digunakan untuk membaca input dari keyboard. Bagian ini sangat penting karena menjadi pusat penyimpanan data yang dipakai oleh seluruh method dalam program.

3.3. Penjelasan Load dan Save Data

```
26 // =====
27 // LOAD DATA DARI FILE TXT
28 // =====
29 static void loadData() {
30
31     try {
32
33         BufferedReader br =
34             new BufferedReader(
35                 new FileReader(fileName: "tiket.txt"));
36
37         String line;
38
39         while ((line = br.readLine()) != null) {
40
41             String[] p = line.split(regex: ",");
42
43             int id = Integer.parseInt(p[0]);
44             String nama = p[1];
45             String film = p[2];
46             int jumlah = Integer.parseInt(p[3]);
47             boolean deleted = Boolean.parseBoolean(p[4]);
48
49             data[jumlahData] =
50                 new Tiket(id, nama, film, jumlah);
51             data[jumlahData].isDeleted = deleted;
52
53             jumlahData++;
54         }
55
56         br.close();
57
58         System.out.println(x: ">> Data berhasil dimuat dari file.");
59
60     } catch (Exception e) {
61
62         System.out.println(x: ">> File belum ada.");
63     }
64 }
65
66
```

```

67 // =====
68 // SIMPAN DATA KE FILE TXT
69 // =====
70 static void simpanData() {
71
72     try {
73
74         PrintWriter pw =
75             new PrintWriter(
76                 new FileWriter(fileName: "tiket.txt"));
77
78         for (int i = 0; i < jumlahData; i++) {
79
80             pw.println(
81                 data[i].id + "," +
82                 data[i].nama + "," +
83                 data[i].film + "," +
84                 data[i].jumlah + "," +
85                 data[i].isDeleted
86             );
87         }
88
89         pw.close();
90
91         System.out.println(x: ">> Data berhasil disimpan.");
92
93     } catch (Exception e) {
94
95         System.out.println(x: ">> Gagal menyimpan data.");
96     }
97 }

```

Pada method loadData(), program melakukan proses pembacaan file TXT bernama tiket.txt. Program menggunakan BufferedReader dan FileReader untuk membaca file baris demi baris. Setiap baris dipisahkan menggunakan split(",") sehingga data dapat dimasukkan kembali ke dalam object Tiket. Fungsi utama method ini adalah agar data yang sebelumnya sudah disimpan tidak hilang ketika program dijalankan kembali. Jika file belum ada, program akan menampilkan pesan bahwa file belum tersedia. Method simpanData() digunakan untuk menyimpan seluruh data tiket ke dalam file TXT. Program menggunakan PrintWriter dan FileWriter untuk menulis data ke file. Seluruh isi array akan dituliskan dalam format yang dipisahkan dengan tanda koma. Tujuan method ini adalah membuat data menjadi permanen sehingga tetap tersimpan walaupun program ditutup.

3.4. Penjelasan Data Awal dan Main Program & Penjelasan Fitur Tambah dan Tampil Data.

```

99 // =====
100 // DATA AWAL
101 // =====
102 static void muatDataAwal() {
103
104 data[jumlahData++] = new Tiket(id: 101, nama: "Antony", film: "Ghost in the Cell", jumlah: 1);
105 data[jumlahData++] = new Tiket(id: 102, nama: "Bagaskara", film: "Ghost in the Cell", jumlah: 2);
106 data[jumlahData++] = new Tiket(id: 103, nama: "Bruno", film: "Ghost in the Cell", jumlah: 4);
107 data[jumlahData++] = new Tiket(id: 104, nama: "Joko Anwar", film: "Ghost in the Cell", jumlah: 1);
108 data[jumlahData++] = new Tiket(id: 105, nama: "Tokek", film: "Ghost in the Cell", jumlah: 2);
109 data[jumlahData++] = new Tiket(id: 106, nama: "Six", film: "Ghost in the Cell", jumlah: 3);
110
111 data[jumlahData++] = new Tiket(id: 201, nama: "Risa Adistia", film: "Danur 2", jumlah: 2);
112 data[jumlahData++] = new Tiket(id: 202, nama: "Peter", film: "Danur 2", jumlah: 1);
113 data[jumlahData++] = new Tiket(id: 203, nama: "William", film: "Danur 2", jumlah: 3);
114 data[jumlahData++] = new Tiket(id: 204, nama: "Janshen", film: "Danur 2", jumlah: 2);
115 data[jumlahData++] = new Tiket(id: 205, nama: "Hans", film: "Danur 2", jumlah: 4);
116 data[jumlahData++] = new Tiket(id: 206, nama: "Asih", film: "Danur 2", jumlah: 1);
117
118 data[jumlahData++] = new Tiket(id: 301, nama: "Rini Suwono", film: "Pengabdi Setan 3", jumlah: 4);
119 data[jumlahData++] = new Tiket(id: 302, nama: "Tony Suwono", film: "Pengabdi Setan 3", jumlah: 2);
120 data[jumlahData++] = new Tiket(id: 303, nama: "Hendra Suwono", film: "Pengabdi Setan 3", jumlah: 3);
121 data[jumlahData++] = new Tiket(id: 304, nama: "Bondi Suwono", film: "Pengabdi Setan 3", jumlah: 1);
122 data[jumlahData++] = new Tiket(id: 305, nama: "Uwak", film: "Pengabdi Setan 3", jumlah: 5);
123 data[jumlahData++] = new Tiket(id: 306, nama: "Budiman", film: "Pengabdi Setan 3", jumlah: 2);
124
125 data[jumlahData++] = new Tiket(id: 401, nama: "Alana Shinta", film: "Sri Asih", jumlah: 3);
126 data[jumlahData++] = new Tiket(id: 402, nama: "Sari Dewi", film: "Sri Asih", jumlah: 2);
127 data[jumlahData++] = new Tiket(id: 403, nama: "Komandan Wira", film: "Sri Asih", jumlah: 4);
128 data[jumlahData++] = new Tiket(id: 404, nama: "Nadia Permata", film: "Sri Asih", jumlah: 1);
129 data[jumlahData++] = new Tiket(id: 405, nama: "Farhan Malik", film: "Sri Asih", jumlah: 3);
130 data[jumlahData++] = new Tiket(id: 406, nama: "Dewi Ratna", film: "Sri Asih", jumlah: 2);
131
132 data[jumlahData++] = new Tiket(id: 501, nama: "Sancaka", film: "Gundala", jumlah: 5);
133 data[jumlahData++] = new Tiket(id: 502, nama: "Pengkor", film: "Gundala", jumlah: 2);
134 data[jumlahData++] = new Tiket(id: 503, nama: "Wulan", film: "Gundala", jumlah: 3);
135 data[jumlahData++] = new Tiket(id: 504, nama: "Ghani", film: "Gundala", jumlah: 1);
136 data[jumlahData++] = new Tiket(id: 505, nama: "Ridwan Bahri", film: "Gundala", jumlah: 4);
137 data[jumlahData++] = new Tiket(id: 506, nama: "Swara Bumi", film: "Gundala", jumlah: 2);
138
139 System.out.println(x: ">> Data awal berhasil dimuat.");
140 }

```

```

142 // =====
143 // MAIN
144 // =====
145 Run | Debug
146 public static void main(String[] args) {
147
148 // LOAD DATA FILE
149 loadData();
150
151 // Kalau file kosong
152 if (jumlahData == 0) {
153 muatDataAwal();
154 }
155
156 int menu;
157
158 do {
159
160 System.out.println(x: "\n=====");
161 System.out.println(x: "SISTEM TIKET BIOSKOP");
162 System.out.println(x: "=====");
163 System.out.println(x: "| No | Menu |");
164 System.out.println(x: "=====");
165 System.out.println(x: "| 1 | Tambah Tiket |");
166 System.out.println(x: "| 2 | Tampil Semua Data |");
167 System.out.println(x: "| 3 | Urutkan Nama |");
168 System.out.println(x: "| 4 | Urutkan ID |");
169 System.out.println(x: "| 5 | Urutkan Tiket Terbanyak |");
170 System.out.println(x: "| 6 | Cari ID |");
171 System.out.println(x: "| 7 | Cari Nama |");
172 System.out.println(x: "| 8 | Cari Film |");
173 System.out.println(x: "| 9 | Hapus Tiket |");
174 System.out.println(x: "| 10 | Statistik |");
175 System.out.println(x: "| 11 | Edit Data |");
176 System.out.println(x: "| 12 | Update Status |");
177 System.out.println(x: "| 13 | Counter Data |");
178 System.out.println(x: "| 14 | Simpan Data ke TXT |");
179 System.out.println(x: "| 0 | Keluar |");
180 System.out.println(x: "=====");
181
182 System.out.print(s: "Pilih Menu : ");
183 menu = sc.nextInt();

```

```

184         switch(menu) {
185
186             case 1:
187                 tambah();
188                 break;
189
190             case 2:
191                 tampil();
192                 break;
193
194             case 3:
195                 urutNama();
196                 break;
197
198             case 4:
199                 urutID();
200                 break;
201
254         // =====
255         // TAMBAH DATA
256         // =====
257         static void tambah() {
258
259             System.out.print(s: "ID : ");
260             int id = sc.nextInt();
261             sc.nextLine();
262
263             System.out.print(s: "Nama Pemesan : ");
264             String nama = sc.nextLine();
265
266             System.out.print(s: "Judul Film : ");
267             String film = sc.nextLine();
268
269             System.out.print(s: "Jumlah Tiket : ");
270             int jumlah = sc.nextInt();
271
272             data[jumlahData++] =
273                 new Tiket(id, nama, film, jumlah);
274
275             System.out.println(x: ">> Data berhasil ditambahkan.");
276         }
278         // =====
279         // TAMPIL DATA
280         // =====
281         static void tampil() {
282
283             System.out.println(x: "\n=====");
284             System.out.printf(format: "%-5s | %-25s | %-25s | %-10s | %-15s | \n",
285                 ...args: "ID", "Nama Pemesan", "Judul Film", "Jumlah", "Status");
286             System.out.println(x: "=====");
287
288             for (int i = 0; i < jumlahData; i++) {
289
290                 String status =
291                     data[i].isDeleted ? "Nonaktif" : "Aktif";
292
293                 System.out.printf(format: "%-5d | %-25s | %-25s | %-10d | %-15s | \n",
294                     data[i].id,
295                     data[i].nama,
296                     data[i].film,
297                     data[i].jumlah,
298                     status);
299             }
300
301             System.out.println(x: "=====");
302         }

```

Program menggunakan object Tiket untuk mengisi array dengan data tiket awal pada bagian muat DataAwal(). Metode ini akan digunakan

jika file TXT masih kosong atau tidak ada. Program dapat diuji langsung dengan data awal tanpa harus memasukkan data satu per satu terlebih dahulu. Pusat jalan program adalah bagian main(). Dalam metode ini, program pertama kali memanggil loadData() untuk membaca data dari file dan memanggil muatDataAwal jika data tidak ada. Selanjutnya, program menggunakan perulangan "do while" untuk menampilkan menu. Ini membuat menu tetap muncul sampai pengguna memilih keluar. Bagian ini juga memiliki switch case yang digunakan untuk menjalankan fitur yang dipilih oleh user, seperti menambah, menampilkan, menyortir, mencari, mengedit, menghapus, dan statistik. Data tiket baru dapat ditambahkan ke array dengan menggunakan metode tambah(). User Diminta agar memasukkan identitas mereka, nama pemesan, judul film, dan jumlah tiket. Program membuat object Tiket baru dan menyimpannya ke array data setelah input selesai. Variabel jumlahData kemudian bertambah satu untuk menunjukkan bahwa data baru telah masuk. Metode tampil() memungkinkan program untuk menampilkan seluruh data tiket dalam bentuk tabel. Program melakukan perulangan ke seluruh isi array dan mencetak informasi seperti ID, nama, film, jumlah tiket, dan status data; nilai isDeleted menentukan status: jika bernilai true maka status menjadi nonaktif, sedangkan jika bernilai false maka status menjadi aktif. Metode ini memudahkan pengguna untuk melihat semua data yang disimpan di sistem.

3.5. Penjelasan Sorting dan Searching

```

304 // =====
305 // SORT NAMA
306 // =====
307 static void urutNama() {
308
309     for (int i = 0; i < jumlahData - 1; i++) {
310
311         int minIdx = i;
312
313         for (int j = i + 1; j < jumlahData; j++) {
314
315             if (data[j].nama.compareToIgnoreCase(data[minIdx].nama) < 0) {
316                 minIdx = j;
317             }
318
319         }
320
321         Tiket temp = data[minIdx];
322         data[minIdx] = data[i];
323         data[i] = temp;
324     }
325
326     System.out.println(x: ">> Nama berhasil diurutkan.");
327 }

```

```

328 // =====
329 // SORT ID
330 // =====
331 static void urutID() {
332
333     for (int i = 0; i < jumlahData - 1; i++) {
334
335         for (int j = 0; j < jumlahData - i - 1; j++) {
336
337             if (data[j].id > data[j + 1].id) {
338
339                 Tiket temp = data[j];
340                 data[j] = data[j + 1];
341                 data[j + 1] = temp;
342             }
343         }
344     }

```

```

349 // =====
350 // SORT TIKET TERBANYAK
351 // =====
352 static void urutTerbanyak() {
353
354     for (int i = 0; i < jumlahData - 1; i++) {
355
356         for (int j = 0; j < jumlahData - i - 1; j++) {
357
358             if (data[j].jumlah < data[j + 1].jumlah) {
359
360                 Tiket temp = data[j];
361                 data[j] = data[j + 1];
362                 data[j + 1] = temp;
363             }
364         }
365     }
366
367     System.out.println(x: ">> Data berhasil diurutkan.");
368 }

```

```

370 // =====
371 // BINARY SEARCH
372 // =====
373 static void cariID() {
374
375     urutID();
376
377     System.out.print(s: "Cari ID : ");
378     int cari = sc.nextInt();
379
380     int low = 0;
381     int high = jumlahData - 1;
382
383     while (low <= high) {
384
385         int mid = (low + high) / 2;
386
387         if (data[mid].id == cari) {
388
389             System.out.println(x: "\n=====");
390             System.out.println(x: "|          DATA DITEMUKAN          |");
391             System.out.println(x: "=====");
392
393             System.out.printf(format: "| %-15s : %-32d |\n", ...args: "ID", data[mid].id);
394             System.out.printf(format: "| %-15s : %-32s |\n", ...args: "Nama", data[mid].nama);
395             System.out.printf(format: "| %-15s : %-32s |\n", ...args: "Film", data[mid].film);
396             System.out.printf(format: "| %-15s : %-32d |\n", ...args: "Jumlah", data[mid].jumlah);
397
398             System.out.println(x: "=====");
399
400             return;
401         }
402
403         if (data[mid].id < cari)
404             low = mid + 1;
405         else
406             high = mid - 1;
407     }
408
409     System.out.println(x: "Data tidak ditemukan.");
410 }

```

```

412 // =====
413 // LINEAR SEARCH
414 // =====
415 static void cariNama() {
416
417     sc.nextLine();
418
419     System.out.print(s: "Cari Nama : ");
420     String cari = sc.nextLine();
421
422     for (int i = 0; i < jumlahData; i++) {
423
424         if (data[i].nama.equalsIgnoreCase(cari)
425             && !data[i].isDeleted) {
426
427             System.out.println(x: "\n=====");
428             System.out.printf(format: "| %-5s | %-25s | %-20s | %-10s |\n",
429                 ...args: "ID", "Nama", "Film", "Jumlah");
430             System.out.println(x: "=====");
431
432             System.out.printf(format: "| %-5d | %-25s | %-20s | %-10d |\n",
433                 data[i].id,
434                 data[i].nama,
435                 data[i].film,
436                 data[i].jumlah);
437
438             System.out.println(x: "=====");
439
440             return;
441         }
442     }
443
444     System.out.println(x: "Nama tidak ditemukan.");
445 }

```

Pada bagian sorting dan searching, program menggunakan beberapa algoritma untuk mengurutkan dan mencari data. Method `urutNama()` menggunakan algoritma Selection Sort untuk mengurutkan data berdasarkan nama secara alfabet. Algoritma ini bekerja dengan mencari data terkecil pada setiap iterasi lalu menukarnya dengan posisi awal. Kemudian method `urutID()` dan `urutTerbanyak()` menggunakan algoritma Bubble Sort. Pada `urutID()`, data diurutkan berdasarkan ID dari kecil ke besar, sedangkan pada `urutTerbanyak()` data diurutkan berdasarkan jumlah tiket terbanyak. Bubble Sort bekerja dengan membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Untuk searching, method `cariID()` menggunakan Binary Search karena data sudah diurutkan terlebih dahulu berdasarkan ID. Binary Search bekerja dengan membagi data menjadi dua bagian dan mencari posisi tengah sehingga pencarian lebih cepat dibanding mencari satu per satu. Sedangkan method `cariNama()` dan `cariFilm()` menggunakan Linear Search, yaitu mencari data secara berurutan dari awal array sampai data ditemukan. Linear Search cocok digunakan untuk pencarian nama dan film karena data belum tentu terurut berdasarkan atribut tersebut.

3.6. Penjelasan Hapus Data

```
486 // =====
487 // HAPUS DATA (HARD DELETE)
488 // =====
489 static void hapus() {
490
491     System.out.print(s: "Masukkan ID yang ingin dihapus : ");
492     int id = sc.nextInt();
493
494     boolean ditemukan = false;
495
496     for (int i = 0; i < jumlahData; i++) {
497
498         if (data[i].id == id) {
499
500             // Geser semua data ke kiri
501             for (int j = i; j < jumlahData - 1; j++) {
502
503                 data[j] = data[j + 1];
504             }
505
506             // Kosongkan data terakhir
507             data[jumlahData - 1] = null;
508
509             // Kurangi jumlah data
510             jumlahData--;
511
512             ditemukan = true;
513
514             System.out.println(x: ">> Data berhasil dihapus permanen.");
515
516             break;
517         }
518     }
519
520     if (!ditemukan) {
521
522         System.out.println(x: ">> ID tidak ditemukan.");
523     }
524 }
```

Method `hapus()` digunakan untuk menghapus data secara permanen atau hard delete. Program akan mencari ID yang dimasukkan user, kemudian seluruh data setelah posisi tersebut digeser ke kiri agar tidak ada data kosong di tengah array. Setelah itu elemen terakhir dikosongkan dan `jumlahData` dikurangi satu. Cara ini membuat data benar-benar hilang dari array.

3.7. Penjelasan Statistik dan Counter Data

```
526 // =====
527 // STATISTIK
528 // =====
529 static void statistik() {
530
531     int aktif = 0;
532     int totalTiket = 0;
533
534     for (int i = 0; i < jumlahData; i++) {
535
536         if (!data[i].isDeleted) {
537
538             aktif++;
539             totalTiket += data[i].jumlah;
540         }
541     }
542
543     System.out.println(x: "\n=====");
544     System.out.println(x: "|                                STATISTIK                                |");
545     System.out.println(x: "=====");
546
547     System.out.printf(format: "| %-30s | %-17d |\n",
548         ...args: "Total Data Aktif", aktif);
549
550     System.out.printf(format: "| %-30s | %-17d |\n",
551         ...args: "Total Tiket", totalTiket);
552
553     System.out.println(x: "=====");
554 }
555
556
557
558
559
560
561
562
563 // =====
564 // COUNTER DATA
565 // =====
566 static void tampilCounter() {
567
568     int aktif = 0;
569     int nonaktif = 0;
570
571     for (int i = 0; i < jumlahData; i++) {
572
573         if (data[i].isDeleted)
574             nonaktif++;
575         else
576             aktif++;
577     }
578
579     System.out.println(x: "\n=====");
580     System.out.println(x: "|                                COUNTER DATA ARRAY                                |");
581     System.out.println(x: "=====");
582
583     System.out.printf(format: "| %-30s | %-17d |\n",
584         ...args: "Kapasitas Maksimal", data.length);
585
586     System.out.printf(format: "| %-30s | %-17d |\n",
587         ...args: "Total Slot Terisi", jumlahData);
588
589     System.out.printf(format: "| %-30s | %-17d |\n",
590         ...args: "Data Aktif", aktif);
591
592     System.out.printf(format: "| %-30s | %-17d |\n",
593         ...args: "Data Nonaktif", nonaktif);
594
595     System.out.printf(format: "| %-30s | %-17d |\n",
596         ...args: "Sisa Slot Kosong", (data.length - jumlahData));
597
598     System.out.println(x: "=====");
599 }
600 }
```

Method statistik() digunakan untuk menghitung jumlah data aktif dan total tiket yang masih aktif. Program melakukan perulangan pada seluruh data lalu menjumlahkan tiket dari data yang statusnya aktif. Hasil akhirnya ditampilkan dalam bentuk tabel statistik. Method selanjutnya yaitu tampilCounter() digunakan untuk menampilkan

informasi kapasitas array, jumlah slot yang sudah terisi, jumlah data aktif, jumlah data nonaktif, dan sisa slot kosong. Method ini membantu user mengetahui kondisi penggunaan array dalam program.

3.8. Penjelasan Edit dan Update Status

```
556 // =====
557 // EDIT DATA
558 // =====
559 static void edit() {
560
561     System.out.print(s: "Masukkan ID yang ingin diedit : ");
562     int idEdit = sc.nextInt();
563     sc.nextLine();
564
565     for (int i = 0; i < jumlahData; i++) {
566
567         if (data[i].id == idEdit && !data[i].isDeleted) {
568
569             System.out.print(s: "Nama Baru : ");
570             data[i].nama = sc.nextLine();
571
572             System.out.print(s: "Film Baru : ");
573             data[i].film = sc.nextLine();
574
575             System.out.print(s: "Jumlah Baru : ");
576             data[i].jumlah = sc.nextInt();
577
578             System.out.println(x: ">> Data berhasil diedit.");
579
580             return;
581         }
582     }
583
584     System.out.println(x: "ID tidak ditemukan.");
585 }

```

```
587 // =====
588 // UPDATE STATUS
589 // =====
590 static void updateStatus() {
591
592     System.out.print(s: "Masukkan ID : ");
593     int id = sc.nextInt();
594
595     for (int i = 0; i < jumlahData; i++) {
596
597         if (data[i].id == id) {
598
599             data[i].isDeleted = !data[i].isDeleted;
600
601             String status =
602                 data[i].isDeleted ? "Nonaktif" : "Aktif";
603
604             System.out.println(">> Status berhasil diubah menjadi " + status);
605
606             return;
607         }
608     }
609
610     System.out.println(x: "ID tidak ditemukan.");
611 }

```

Method edit() berfungsi mengubah data tiket yang sudah ada. User memasukkan ID data yang ingin diedit, lalu program mencari data tersebut dan mengganti nama, film, serta jumlah tiket sesuai input

baru. Fitur ini berguna jika terjadi kesalahan input data sebelumnya. Method `updateStatus()` digunakan untuk mengubah status data menjadi aktif atau nonaktif tanpa menghapus datanya. Program membalik nilai boolean `isDeleted` menggunakan operator. Jika sebelumnya aktif maka menjadi nonaktif, dan sebaliknya. Fitur ini berguna untuk menonaktifkan data sementara tanpa kehilangan isi datanya.

BAB IV

ANALISIS KOMPLEKSITAS

4.1. Analisis Kompleksitas Program

Analisis kompleksitas digunakan untuk mengetahui efisiensi algoritma dalam suatu program berdasarkan waktu proses (*time complexity*) dan penggunaan memori (*space complexity*). Pada program Sistem Tiket Bioskop ini, sebagian besar proses menggunakan array, looping, sorting, dan searching sehingga performanya dipengaruhi oleh jumlah data (n) yang tersimpan.

Kompleksitas dianalisis dalam tiga kondisi, yaitu:

- **Best Case** → kondisi terbaik saat proses berjalan paling cepat.
- **Average Case** → kondisi rata-rata saat program dijalankan.
- **Worst Case** → kondisi terburuk saat proses berjalan paling lambat.

4.2. Tabel Analisis Kompleksitas

No.	Fitur	Algoritma	Best Case	Average Case	Worst Case	Space Complexity	Penjelasan
1.	tambah()	Insert Array	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Data langsung ditambahkan ke indeks terakhir array tanpa looping.
2.	tampil()	Transversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Semua data ditampilkan satu per satu menggunakan perulangan.
3.	urutNama()	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Membandingkan setiap elemen nama dengan elemen

							lainnya.
4.	urutID()	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Mengurutkan ID dengan pertukaran data berulang.
5.	urutTerbanyak()	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Mengurutkan jumlah tiket terbesar ke terkecil.
6.	cariID()	Bubble Sort & Binary Search	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Data dicari dengan membagi array menjadi dua bagian.
7.	cariNama()	Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Nama dicari satu per satu dari awal array.
8.	cariFilm()	Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Judul film diperiksa satu per satu hingga ditemukan.
9.	hapus()	Delete & Shift Array	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Setelah data ditemukan, elemen array digeser ke kiri.
10.	statistik()	Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Menghitung total data aktif dan total tiket.
11.	edit()	Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Sistem mencari ID terlebih dahulu sebelum mengedit data.
12.	updateStatus()	Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Sistem mencari ID lalu mengubah status data.
13.	tampilCounter()	Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Menghitung jumlah data aktif dan nonaktif.
14.	loadData()	Read File	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Membaca seluruh isi file TXT ke

							array.
15.	SimpanData()	Write File	O(n)	O(n)	O(n)	O(1)	Menulis seluruh data array ke file TXT.

4.3. Penjelasan Kompleksitas

4.3.1. Kompleksitas Penambahan Data

Method `tambah()` memiliki kompleksitas waktu $O(1)$ karena data baru langsung dimasukkan ke indeks terakhir array tanpa pencarian ataupun perulangan.

```
data[jumlahData++] =
new Tiket(id, nama, film, jumlah);
```

Kompleksitas ruangnya juga $O(1)$ karena hanya membutuhkan satu objek baru setiap penambahan data.

4.3.2. Kompleksitas Pengurutan Data

4.3.2.1. Selection Sort pada `urutNama()`

Method `urutNama()` menggunakan algoritma Selection Sort. Algoritma ini selalu membandingkan seluruh elemen untuk mencari nilai minimum sehingga kompleksitasnya tetap $O(n^2)$ pada semua kondisi.

```
for (int i = 0; i < jumlahData - 1; i++) {
    int minIdx = i;

    for (int j = i + 1; j < jumlahData; j++) {
```

Space complexity algoritma ini adalah $O(1)$ karena hanya menggunakan variabel sementara (`temp`) untuk pertukaran data.

4.3.2.2. Bubble Sort pada `urutID()` dan `urutTerbanyak()`

Bubble Sort melakukan proses perbandingan dan pertukaran data secara berulang hingga seluruh data terurut.

Pada program ini, algoritma Bubble Sort belum menggunakan optimasi penghentian awal (*early stop*) seperti variabel `swapped`. Oleh karena itu, meskipun data sudah dalam keadaan terurut, proses perulangan tetap berjalan sampai selesai.

```
for (int i = 0; i < jumlahData - 1; i++) {  
    for (int j = 0; j < jumlahData - i - 1; j++) {
```

4.3.3. Kompleksitas Pencarian Data

4.3.3.1. Binary Search (`cariID()`)

Method `cariID()` menggunakan Binary Search untuk mencari ID tiket. Namun sebelum proses pencarian dilakukan, data terlebih dahulu diurutkan menggunakan method `urutID()` yang memakai algoritma Bubble Sort.

```
urutID();
```

Setelah data terurut, proses pencarian dilakukan menggunakan Binary Search.

```
int mid = (low + high) / 2;
```

Secara teori, Binary Search memiliki kompleksitas $O(\log n)$. Akan tetapi, karena pada implementasi program ini terdapat proses sorting terlebih dahulu dengan Bubble Sort yang memiliki

kompleksitas $O(n^2)$, maka total kompleksitas method `cariID()` didominasi oleh proses sorting sehingga menjadi:

- Best Case = $O(n^2)$
- Average Case = $O(n^2)$
- Worst Case = $O(n^2)$

Space complexity dari method ini adalah $O(1)$ karena hanya menggunakan beberapa variabel tambahan seperti `low`, `high`, dan `mid`.

4.3.3.2. Linear Search (`cariNama()`, `cariFilm()`, `edit()`, `updateStatus()`)

Linear Search memeriksa data satu per satu dari awal array hingga data ditemukan.

Kompleksitas:

- Best Case: $O(1)$ jika data ada di indeks pertama.
- Worst Case: $O(n)$ jika data berada di akhir atau tidak ditemukan.

4.3.4. Kompleksitas Penghapusan Data

Method `hapus()` memiliki kompleksitas $O(n)$ karena setelah data ditemukan, seluruh elemen setelahnya harus digeser ke kiri.

```
for (int j = i; j < jumlahData - 1; j++) {  
    data[j] = data[j + 1];  
}
```

Jika data yang dihapus berada di awal array, maka proses penggeseran akan lebih banyak dibanding data yang berada di akhir array.

4.3.5. Kompleksitas Pembacaan dan Penyimpanan File

4.3.5.1. `loadData()`

Method ini membaca seluruh isi file TXT satu per satu menggunakan looping. Kompleksitas waktunya $O(n)$ karena seluruh data harus diproses. Space complexity sebesar $O(n)$ karena data disimpan kembali ke array objek.

4.3.5.2. `simpanData()`

Method ini menulis seluruh isi array ke file TXT. Kompleksitas waktunya $O(n)$ karena seluruh data ditulis satu per satu. Space complexity tetap $O(1)$ karena tidak membuat struktur data tambahan yang besar.

4.3.6. Analisis Space Complexity

Program menggunakan array objek statis:

```
static Tiket[] data = new Tiket[100];
```

Array tersebut memiliki kapasitas maksimal 100 data tiket. Penggunaan memori utama berasal dari penyimpanan objek `Tiket`.

Selain itu, program hanya menggunakan:

- variabel integer,
- string,
- boolean,
- dan variabel sementara.

Karena tidak menggunakan struktur data dinamis seperti linked list atau tree, penggunaan memori program tergolong ringan dan sederhana.

BAB V

PENGUJIAN

Pada bab ini, dilakukan pengujian terhadap seluruh fitur yang tersedia dalam sistem untuk memastikan fungsi penambahan, pengurutan, pencarian, pengeditan, penghapusan (*hard delete*), pengubahan status (*soft delete*), hingga penyimpanan berkas berjalan sesuai dengan spesifikasi rancangan.

5.1. Tabel Rencana Pengujian (Test Matrix)

ID	Fitur / Skenario Pengujian	Parameter Masukan (Input)	Hasil yang Diharapkan	Status
TC-01	Inisialisasi Data Awal (<i>Load File</i>)	Menjalankan program pertama kali	Sistem membaca file <i>tiket.txt</i> . Jika belum ada, sistem menginisialisasi 30 data penonton awal.	Berhasil
TC-02	Tambah Tiket (Menu 1)	ID: 601, Nama: Gading, Film: Gundala, Jumlah: 3	Data baru tersimpan di akhir array dan kapasitas slot terisi bertambah.	Berhasil

TC-03	Tampil Semua Data (Menu 2)	Memilih Menu 2	Menampilkan seluruh data transaksi dalam format tabel (baik berstatus Aktif maupun Nonaktif).	Berhasil
TC-04	Urutkan Nama (Menu 3)	Memilih Menu 3	Array terurut secara alfabetis (A-Z) berdasarkan nama pemesan (<i>Selection Sort</i>).	Berhasil
TC-05	Urutkan ID (Menu 4)	Memilih Menu 4	Array terurut secara <i>ascending</i> (kecil ke besar) berdasarkan ID Tiket (<i>Bubble Sort</i>).	Berhasil
TC-06	Urutkan Tiket Terbanyak (Menu 5)	Memilih Menu 5	Array terurut secara <i>descending</i> (besar ke kecil) berdasarkan jumlah pembelian tiket (<i>Bubble Sort</i>).	Berhasil
TC-07	Cari ID via <i>Binary</i>	ID Tiket: 205	Mengurutkan ID terlebih dahulu, lalu menampilkan	Berhasil

	<i>Search</i> (Menu 6)		detail data ID 205 secara vertikal.	
TC-08	Cari Nama via <i>Linear Search</i> (Menu 7)	Nama: Sancaka	Menampilkan data tiket atas nama Sancaka dalam format tabel khusus.	Berhasil
TC-09	Cari Film via <i>Linear Search</i> (Menu 8)	Judul Film: Danur 2	Menampilkan seluruh penonton yang membeli tiket film Danur 2.	Berhasil
TC-10	Hapus Tiket Permanen / <i>Hard Delete</i> (Menu 9)	ID Tiket: 103	Menghapus elemen ID 103, menggeser indeks array ke kiri, dan mengurangi jumlah total data.	Berhasil
TC-11	Statistik Penjualan (Menu 10)	Memilih Menu 10	Menampilkan total record data aktif dan akumulasi seluruh tiket yang terjual.	Berhasil
TC-12	Edit Data Tiket (Menu 11)	ID: 101 , Nama Baru, Film Baru, Jumlah Baru	Informasi lama pada ID 101 berhasil diperbarui	Berhasil

			dengan data inputan baru.	
TC-13	Update Status / <i>Soft Delete Toggle</i> (Menu 12)	ID Tiket: 105	Mengubah status ID 105 dari "Aktif" menjadi "Nonaktif" (atau sebaliknya) tanpa menghapus data dari array.	Berhasil
TC-14	Counter Kapasitas Array (Menu 13)	Memilih Menu 13	Menampilkan kalkulasi memori array (Slot Terisi, Slot Kosong, Total Aktif, Total Nonaktif).	Berhasil
TC-15	Simpan File dan Keluar (Menu 14 / 0)	Memilih Menu 14 atau 0	Seluruh perubahan data di dalam array ditulis kembali ke dalam berkas eksternal <u>tiket.txt</u> .	Berhasil

5.2. Dokumentasi Eksekusi Skenario Pengujian

5.2.1. TC-01: Inisialisasi Data Awal (Load File)

Prosedur: Jalankan kelas **Sistemtiketbioskopfixx** melalui Terminal/Command Prompt saat file penyimpanan belum tersedia.

Hasil Output: Sistem menampilkan indikator pesan bahwa file belum ada dan langsung memicu pemuatan 30 data bawaan ke dalam memori.

```
>> Data berhasil dimuat dari file.

=====
|                                     |
|               SISTEM TIKET BIOSKOP               |
|                                     |
|=====|
| No | Menu |
|=====|
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
|=====|
Pilih Menu : 
```

5.2.2. TC-02: Menambahkan Tiket Baru (Menu 1)

Prosedur: Pilih Menu 1. Masukkan input data: ID = 601, Nama = Gading, Judul Film = Gundala, Jumlah Tiket = 3.

Hasil Output: Sistem menampilkan pesan >> Data berhasil ditambahkan..

```
>> Data berhasil dimuat dari file.

=====
|                               SISTEM TIKET BIOSKOP                               |
=====
| No |                               Menu                               |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====

Pilih Menu : 1
ID : 601
Nama Pemesan : Goldan
Judul Film : Agak Laen 2
Jumlah Tiket : 3
>> Data berhasil ditambahkan.
```

5.2.3. TC-03: Menampilkan Seluruh Data (Menu 2)

Prosedur: Pilih Menu **2** langsung setelah skenario penambahan data dijalankan.

Hasil Output: Program menampilkan visualisasi tabel data terstruktur dari indeks awal hingga data baru (ID 601) di posisi paling bawah.

Pilih Menu : 2

ID	Nama Pemesan	Judul Film	Jumlah	Status
101	Antony	Ghost in the Cell	1	Aktif
102	Oji	Ghost in the cell	9	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
106	Six	Ghost in the Cell	3	Aktif
201	Risa Adistia	Danur 2	2	Aktif
202	Peter	Danur 2	1	Aktif
203	William	Danur 2	3	Aktif
204	Janshen	Danur 2	2	Aktif
205	Hans	Danur 2	4	Aktif
206	Asih	Danur 2	1	Aktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
501	Faris	Gundala	7	Aktif
502	Pengkor	Gundala	2	Aktif
503	Wulan	Gundala	3	Aktif
504	Ghani	Gundala	1	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif
601	Goldan	Agak Laen 2	3	Aktif

5.2.4. TC-04: Mengurutkan Nama Pemesan (Menu 3)

Prosedur: Pilih Menu **3** untuk memicu fungsi *Selection Sort*, kemudian verifikasi hasilnya dengan memilih Menu **2**.

Hasil Output: Teks **>> Nama berhasil diurutkan.** muncul. Ketika list data dibuka, urutan baris telah berubah berdasarkan abjad nama pemesan (A-Z) secara *case-insensitive*.

```

=====
|                                     |
|                               SISTEM TIKET BIOSKOP                               |
|                                     |
=====
| No | Menu |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====

Pilih Menu : 3
>> Nama berhasil diurutkan.

```

```

Pilih Menu : 2
=====
| ID | Nama Pemesan | Judul Film | Jumlah | Status |
=====
| 401 | Alana Shinta | Sri Asih | 3 | Aktif |
| 101 | Antony | Ghost in the Cell | 1 | Aktif |
| 206 | Asih | Danur 2 | 1 | Aktif |
| 304 | Bondi Suwono | Pengabdi Setan 3 | 1 | Aktif |
| 103 | Bruno | Ghost in the Cell | 4 | Aktif |
| 306 | Budiman | Pengabdi Setan 3 | 2 | Aktif |
| 406 | Dewi Ratna | Sri Asih | 2 | Aktif |
| 405 | Farhan Malik | Sri Asih | 3 | Aktif |
| 501 | Faris | Gundala | 7 | Aktif |
| 504 | Ghani | Gundala | 1 | Aktif |
| 601 | Goldan | Agak Laen 2 | 3 | Aktif |
| 205 | Hans | Danur 2 | 4 | Aktif |
| 303 | Hendra Suwono | Pengabdi Setan 3 | 3 | Aktif |
| 204 | Janshen | Danur 2 | 2 | Aktif |
| 104 | Joko Anwar | Ghost in the Cell | 1 | Aktif |
| 403 | Komandan Wira | Sri Asih | 4 | Aktif |
| 404 | Nadia Permata | Sri Asih | 1 | Aktif |
| 102 | Oji | Ghost in the cell | 9 | Aktif |
| 502 | Pengkor | Gundala | 2 | Aktif |
| 202 | Peter | Danur 2 | 1 | Aktif |
| 505 | Ridwan Bahri | Gundala | 4 | Nonaktif |
| 301 | Rini Suwono | Pengabdi Setan 3 | 4 | Aktif |
| 201 | Risa Adistia | Danur 2 | 2 | Aktif |
| 402 | Sari Dewi | Sri Asih | 2 | Aktif |
| 106 | Six | Ghost in the Cell | 3 | Aktif |
| 105 | Tokek | Ghost in the Cell | 2 | Aktif |
| 302 | Tony Suwono | Pengabdi Setan 3 | 2 | Aktif |
| 305 | Uwak | Pengabdi Setan 3 | 5 | Aktif |
| 203 | William | Danur 2 | 3 | Aktif |
| 503 | Wulan | Gundala | 3 | Aktif |
=====

```

5.2.5. TC-05: Mengurutkan ID Tiket (Menu 4)

Prosedur: Pilih Menu **4** untuk memicu fungsi *Bubble Sort Ascending*, kemudian verifikasi hasilnya dengan memilih Menu **2**.

Hasil Output: Teks **>> ID berhasil diurutkan.** muncul. Baris data di dalam tabel kembali tersusun rapi berdasarkan nomor ID terkecil ke terbesar.

SISTEM TIKET BIOSKOP				
No	Menu			
1	Tambah Tiket			
2	Tampil Semua Data			
3	Urutkan Nama			
4	Urutkan ID			
5	Urutkan Tiket Terbanyak			
6	Cari ID			
7	Cari Nama			
8	Cari Film			
9	Hapus Tiket			
10	Statistik			
11	Edit Data			
12	Update Status			
13	Counter Data			
14	Simpan Data ke TXT			
0	Keluar			

Pilih Menu : 4
>> ID berhasil diurutkan.

Pilih Menu : 2

ID	Nama Pemesan	Judul Film	Jumlah	Status
101	Antony	Ghost in the Cell	1	Aktif
102	Oji	Ghost in the cell	9	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
106	Six	Ghost in the Cell	3	Aktif
201	Risa Adistia	Danur 2	2	Aktif
202	Peter	Danur 2	1	Aktif
203	William	Danur 2	3	Aktif
204	Janshen	Danur 2	2	Aktif
205	Hans	Danur 2	4	Aktif
206	Asih	Danur 2	1	Aktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
501	Faris	Gundala	7	Aktif
502	Pengkor	Gundala	2	Aktif
503	Wulan	Gundala	3	Aktif
504	Ghani	Gundala	1	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif
601	Goldan	Agak Laen 2	3	Aktif

5.2.6. TC-06: Mengurutkan Tiket Terbanyak (Menu 5)

Prosedur: Pilih Menu **5** untuk memicu fungsi *Bubble Sort Descending*, kemudian verifikasi hasilnya dengan memilih Menu **2**.

Hasil Output: Teks **>> Data berhasil diurutkan.** muncul. Baris paling atas tabel kini ditempati oleh penonton dengan kuantitas pembelian tiket paling banyak.

```
=====
|                                     |
|               SISTEM TIKET BIOSKOP |
|                                     |
|=====|
| No | Menu |
|=====|
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
|=====|
Pilih Menu : 5
>> Data berhasil diurutkan.
```


Pilih Menu : 2

ID	Nama Pemesan	Judul Film	Jumlah	Status
102	Oji	Ghost in the cell	9	Aktif
501	Faris	Gundala	7	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
205	Hans	Danur 2	4	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
601	Goldan	Agak Laen 2	3	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
106	Six	Ghost in the Cell	3	Aktif
203	William	Danur 2	3	Aktif
503	Wulan	Gundala	3	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
204	Janshen	Danur 2	2	Aktif
502	Pengkor	Gundala	2	Aktif
201	Risa Adistia	Danur 2	2	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
101	Antony	Ghost in the Cell	1	Aktif
206	Asih	Danur 2	1	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
504	Ghani	Gundala	1	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
202	Peter	Danur 2	1	Aktif

5.2.7. TC-07: Pencarian ID Tiket via Binary Search (Menu 6)

Prosedur: Pilih Menu 6. Masukkan ID target yang dicari: 205.

Hasil Output: Sistem otomatis mengurutkan ID terlebih dahulu, melakukan pencarian dengan membagi ruang data (*binary split*), lalu menampilkan data spesifik atas nama "Hans" secara vertikal.

```
=====
|                                     |
|                               SISTEM TIKET BIOSKOP                               |
|                                     |
|=====|=====|
| No | Menu |
|=====|=====|
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
|=====|=====|

Pilih Menu : 6
>> ID berhasil diurutkan.
Cari ID : 601

=====
|                                     |
|                               DATA DITEMUKAN                               |
|                                     |
|=====|=====|
| ID      : 601 |
| Nama    : Goldan |
| Film    : Agak Laen 2 |
| Jumlah  : 3 |
|=====|=====|
```

5.2.8. TC-08: Pencarian Nama via Linear Search (Menu 7)

Prosedur: Pilih Menu **7**. Masukkan Nama target yang dicari: **Sancaka**.

Hasil Output: Sistem menelusuri array dari indeks 0 hingga akhir (*linear*), lalu menampilkan satu baris data Sancaka di dalam format sub-tabel pencarian jika data tersebut berstatus aktif.

```

=====
|                                     SISTEM TIKET BIOSKOP                                     |
=====
| No |                                     Menu                                     |
=====
| 1  | Tambah Tiket |
| 2  | Tampil Semua Data |
| 3  | Urutkan Nama |
| 4  | Urutkan ID |
| 5  | Urutkan Tiket Terbanyak |
| 6  | Cari ID |
| 7  | Cari Nama |
| 8  | Cari Film |
| 9  | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0  | Keluar |
=====
Pilih Menu : 7
Cari Nama : Goldan

=====
| ID | Nama | Film | Jumlah |
=====
| 601 | Goldan | Agak Laen 2 | 3 |
=====

```

5.2.9. TC-09: Pencarian Multi-Data Berdasarkan Judul Film (Menu 8)

Prosedur: Pilih Menu 8. Masukkan nama film target: **Danur 2.**

Hasil Output: Sistem menyaring seluruh isi array dan menampilkan semua data penonton yang memiliki kecocokan judul film "Danur 2" ke dalam bentuk tabel kolektif.

```

=====
|          SISTEM TIKET BIOSKOP          |
=====
| No |          Menu          |
=====
| 1 | Tambah Tiket          |
| 2 | Tampil Semua Data     |
| 3 | Urutkan Nama          |
| 4 | Urutkan ID            |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID               |
| 7 | Cari Nama             |
| 8 | Cari Film             |
| 9 | Hapus Tiket           |
| 10 | Statistik             |
| 11 | Edit Data             |
| 12 | Update Status         |
| 13 | Counter Data          |
| 14 | Simpan Data ke TXT    |
| 0 | Keluar                |
=====
Pilih Menu : 8
Masukkan Judul Film : Agak Laen 2

=====
| ID | Nama | Film | Jumlah |
=====
| 601 | Goldan | Agak Laen 2 | 3 |
=====

```

5.2.10. TC-10: Penghapusan Elemen Permanen / Hard Delete (Menu 9)

Prosedur: Pilih Menu 9. Masukkan nomor ID penonton yang ingin dihapus total dari memori: 103 (Tiket Bruno). Setelah sukses, periksa isi tabel menggunakan Menu 2.

Hasil Output: Muncul konfirmasi >> Data berhasil dihapus permanen.. Pada tabel Menu 2, baris data ID 103 telah hilang sepenuhnya karena sistem melakukan pergeseran indeks (*index shifting*) ke kiri untuk menutup kekosongan array.

```

=====
|                                     SISTEM TIKET BIOSKOP                                     |
=====
| No |                                     Menu                                     |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====

Pilih Menu : 9
Masukkan ID yang ingin dihapus : 601
>> Data berhasil dihapus permanen.

```

ID	Nama Pemesan	Judul Film	Jumlah	Status
101	Antony	Ghost in the Cell	1	Aktif
102	Oji	Ghost in the cell	9	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
106	Six	Ghost in the Cell	3	Aktif
201	Risa Adistia	Danur 2	2	Aktif
202	Peter	Danur 2	1	Aktif
203	William	Danur 2	3	Aktif
204	Janshen	Danur 2	2	Aktif
205	Hans	Danur 2	4	Aktif
206	Asih	Danur 2	1	Aktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
501	Faris	Gundala	7	Aktif
502	Pengkor	Gundala	2	Aktif
503	Wulan	Gundala	3	Aktif
504	Ghani	Gundala	1	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif

5.2.11. TC-11: Menampilkan Hasil Statistik Penjualan (Menu 10)

Prosedur: Pilih Menu **10** pada panel pilihan menu utama.

Hasil Output: Program menyajikan kotak informasi ringkas yang menjumlahkan total record data penonton yang aktif beserta akumulasi kuantitas lembar tiket riil yang berhasil terjual.

```
=====
|                                     |
|                               SISTEM TIKET BIOSKOP                               |
|                                     |
| No |                               Menu                               |
|-----|-----|
| 1 |  Tambah Tiket                               |
| 2 |  Tampil Semua Data                         |
| 3 |  Urutkan Nama                             |
| 4 |  Urutkan ID                               |
| 5 |  Urutkan Tiket Terbanyak                  |
| 6 |  Cari ID                                  |
| 7 |  Cari Nama                               |
| 8 |  Cari Film                               |
| 9 |  Hapus Tiket                             |
| 10 | Statistik                               |
| 11 | Edit Data                               |
| 12 | Update Status                           |
| 13 | Counter Data                           |
| 14 | Simpan Data ke TXT                      |
| 0 |  Keluar                                  |
|-----|-----|
Pilih Menu : 10

=====
|                                     |
|                               STATISTIK                               |
|-----|-----|
| Total Data Aktif | 28 |
| Total Tiket      | 78 |
|-----|-----|
=====
```

5.2.12. TC-12: Mengubah Informasi Tiket / Edit Data (Menu 11)

Prosedur: Pilih Menu **11**. Masukkan ID target: **504**.

Masukkan variabel perubahan baru -> Nama: **Tata**, Film: **Biru Baru Movie**, Jumlah: **5**. Buka Menu **2** untuk verifikasi.

Hasil Output: Muncul konfirmasi **>> Data berhasil diedit.** dan baris objek pada indeks tersebut sukses diperbarui dengan informasi terbaru.

```
=====
|                                     SISTEM TIKET BIOSKOP                                     |
=====
| No | Menu |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====
Pilih Menu : 11
Masukkan ID yang ingin diedit : 504
Nama Baru : Tata
Film Baru : Biru Baru Movie
Jumlah Baru : 5
>> Data berhasil diedit.
```

Pilih Menu : 2

ID	Nama Pemesan	Judul Film	Jumlah	Status
101	Antony	Ghost in the Cell	1	Aktif
102	Oji	Ghost in the cell	9	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
106	Six	Ghost in the Cell	3	Aktif
201	Risa Adistia	Danur 2	2	Aktif
202	Peter	Danur 2	1	Aktif
203	William	Danur 2	3	Aktif
204	Janshen	Danur 2	2	Aktif
205	Hans	Danur 2	4	Aktif
206	Asih	Danur 2	1	Aktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
501	Faris	Gundala	7	Aktif
502	Pengkor	Gundala	2	Aktif
503	Wulan	Gundala	3	Aktif
504	Tata	Biru Baru Movie	5	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif

5.2.13. TC-13: Manajemen Flag Soft Delete / Update Status (Menu 12)

Prosedur: Pilih Menu 12. Masukkan ID target: 105 (Tiket atas nama Tokek). Setelah muncul notifikasi, buka Menu 2 untuk meninjau perubahan status.

Hasil Output: Sistem menampilkan pesan >> Status berhasil diubah menjadi Nonaktif. Ketika tabel Menu 2 dibuka, objek data ID 105 masih berada di dalam array namun kolom statusnya berubah menjadi Nonaktif. Jika Menu 12 dipanggil kembali pada ID yang sama, status akan berbalik (*toggle*) kembali menjadi Aktif.


```

=====
|                                     |
|                               SISTEM TIKET BIOSKOP                               |
|                                     |
=====
| No | Menu |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====

Pilih Menu : 9
Masukkan ID yang ingin dihapus : 601
>> Data berhasil dihapus permanen.

```

ID	Nama Pemesan	Judul Film	Jumlah	Status
101	Antony	Ghost in the Cell	1	Aktif
102	Oji	Ghost in the cell	9	Aktif
103	Bruno	Ghost in the Cell	4	Aktif
104	Joko Anwar	Ghost in the Cell	1	Aktif
105	Tokek	Ghost in the Cell	2	Aktif
106	Six	Ghost in the Cell	3	Aktif
201	Risa Adistia	Danur 2	2	Aktif
202	Peter	Danur 2	1	Aktif
203	William	Danur 2	3	Aktif
204	Janshen	Danur 2	2	Aktif
205	Hans	Danur 2	4	Aktif
206	Asih	Danur 2	1	Aktif
301	Rini Suwono	Pengabdi Setan 3	4	Aktif
302	Tony Suwono	Pengabdi Setan 3	2	Aktif
303	Hendra Suwono	Pengabdi Setan 3	3	Aktif
304	Bondi Suwono	Pengabdi Setan 3	1	Aktif
305	Uwak	Pengabdi Setan 3	5	Aktif
306	Budiman	Pengabdi Setan 3	2	Aktif
401	Alana Shinta	Sri Asih	3	Aktif
402	Sari Dewi	Sri Asih	2	Aktif
403	Komandan Wira	Sri Asih	4	Aktif
404	Nadia Permata	Sri Asih	1	Aktif
405	Farhan Malik	Sri Asih	3	Aktif
406	Dewi Ratna	Sri Asih	2	Aktif
501	Faris	Gundala	7	Aktif
502	Pengkor	Gundala	2	Aktif
503	Wulan	Gundala	3	Aktif
504	Ghani	Gundala	1	Aktif
505	Ridwan Bahri	Gundala	4	Nonaktif

5.2.14. TC-14: Monitoring Kapasitas Memori Array (Menu 13)

Prosedur: Pilih Menu **13** pada baris perintah menu utama.

Hasil Output: Sistem menampilkan detail penggunaan memori statis array, mencakup Kapasitas Maksimal (100 slot), jumlah slot terisi, jumlah data aktif, jumlah data nonaktif (*soft deleted*), serta sisa slot kosong yang masih bisa digunakan untuk menampung data baru.

SISTEM TIKET BIOSKOP	
No	Menu
1	Tambah Tiket
2	Tampil Semua Data
3	Urutkan Nama
4	Urutkan ID
5	Urutkan Tiket Terbanyak
6	Cari ID
7	Cari Nama
8	Cari Film
9	Hapus Tiket
10	Statistik
11	Edit Data
12	Update Status
13	Counter Data
14	Simpan Data ke TXT
0	Keluar

Pilih Menu : 13

COUNTER DATA ARRAY	
Kapasitas Maksimal	100
Total Slot Terisi	29
Data Aktif	29
Data Nonaktif	0
Sisa Slot Kosong	71

5.2.15. TC-15: Terminasi Program dan Sinkronisasi File Eksternal (Menu 0)

Prosedur: Pilih Menu 0 untuk menghentikan program secara reguler. Setelah program tertutup, buka folder direktori project dan periksa dokumen berkas `tiket.txt`.

Hasil Output: Konsol menampilkan pesan `>> Data berhasil disimpan.` disusul dengan penutupan sesi program `>> Program selesai..` File `tiket.txt` berhasil terbuat/terperbarui dengan struktur data berformat CSV dipisahkan oleh tanda koma sesuai kondisi final data di dalam array.

```
>> Data berhasil dimuat dari file.

=====
|                               SISTEM TIKET BIOSKOP                               |
=====
| No |                               Menu                               |
=====
| 1 | Tambah Tiket |
| 2 | Tampil Semua Data |
| 3 | Urutkan Nama |
| 4 | Urutkan ID |
| 5 | Urutkan Tiket Terbanyak |
| 6 | Cari ID |
| 7 | Cari Nama |
| 8 | Cari Film |
| 9 | Hapus Tiket |
| 10 | Statistik |
| 11 | Edit Data |
| 12 | Update Status |
| 13 | Counter Data |
| 14 | Simpan Data ke TXT |
| 0 | Keluar |
=====
Pilih Menu : 0
>> Data berhasil disimpan.
>> Program selesai.
```

BAB VI

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, program Sistem Tiket Bioskop berbasis Java berhasil dikembangkan sesuai tujuan yang direncanakan. Sistem mampu melakukan pengelolaan data tiket bioskop menggunakan struktur data array serta mendukung berbagai operasi seperti tambah data, tampil data, edit data, hapus data, update status, sorting, searching, statistik, dan penyimpanan data ke file teks.

Program ini juga berhasil mengimplementasikan beberapa algoritma dasar, yaitu Selection Sort, Bubble Sort, Binary Search, dan Linear Search. Berdasarkan analisis kompleksitas, algoritma sorting memiliki kompleksitas waktu $O(n^2)$, sedangkan proses searching menggunakan Binary Search memiliki kompleksitas lebih cepat dibanding pencarian linear, meskipun pada implementasinya masih dipengaruhi proses sorting data terlebih dahulu.

Selain itu, fitur penyimpanan data menggunakan file `.txt` memungkinkan data tetap tersimpan meskipun program ditutup. Dengan demikian, sistem yang dibuat telah memenuhi kebutuhan pengelolaan data tiket bioskop sederhana berbasis console serta mampu menerapkan konsep dasar pemrograman Java, struktur data, algoritma, dan file handling secara terintegrasi.